

# Taller de Big Data - Apache Kafka

## Resumen de clase y desarrollo del Práctico 4

CDD7018 · Unidad 3 · Grupo 4

Dataset: Amazon Reviews 2023 - Software

Profesor: Dante Travisany

### Objetivo del documento

Resumir los conceptos principales de la clase de Kafka y dejar trazado, paso a paso, cómo se desarrolló el Práctico 4 del Grupo 4, incluyendo decisiones de diseño, resultados reales, pruebas de consumer groups, lag, at-least-once e idempotencia.

## Resumen ejecutivo

La clase introduce el paso desde procesamiento batch hacia streaming: los datos ya no se consideran un archivo cerrado, sino un flujo continuo de eventos. Kafka actúa como capa distribuida de ingestión, persistencia y distribución; Spark Structured Streaming o Flink se encargan luego del procesamiento avanzado.

En el práctico, el histórico de reseñas de Amazon se convirtió en un stream. Se creó el topic g4.resenas con 6 particiones, replicación 3 y retención de 14 días. El productor publicó 181.675 reseñas desde 2022 y el consumidor calculó métricas por producto, enriqueció con metadatos y generó alertas.

Las pruebas demostraron reparto de particiones, rebalanceo, consumer groups independientes, crecimiento controlado del lag y semántica at-least-once. El resultado final confirma que el consumidor Python no es idempotente y que el estado local es una limitación importante.

# 1. De batch a streaming

## Batch vs. streaming

---

Hasta el Práctico 3 el trabajo estaba centrado en datos en reposo: HDFS, MapReduce, Pig, Hive y Spark sobre conjuntos acotados. En streaming los eventos llegan continuamente y deben procesarse a medida que aparecen.

La diferencia principal es la latencia. En batch se aceptan minutos u horas; en streaming se buscan segundos o milisegundos. Ejemplos típicos son transacciones, sensores IoT, logs y métricas de sistemas.

## ¿Qué resuelve Kafka?

---

Kafka es una plataforma distribuida para manejar streams de eventos. Se comporta como un log distribuido: los productores agregan eventos al final, los datos permanecen almacenados durante una retención configurable y cada consumidor avanza mediante offsets.

Kafka no elimina un mensaje cuando alguien lo lee. Esto permite que varios consumer groups lean el mismo topic de forma independiente y que un grupo pueda volver a leer datos históricos mientras sigan dentro de la retención.

## Arquitectura fundamental

---

Productor: publica eventos. Topic: nombre lógico del flujo. Partición: fragmento físico ordenado del topic. Broker: servidor Kafka que almacena particiones. Consumidor: lee eventos. Consumer group: conjunto de consumidores que se reparten las particiones.

En el clúster del curso existen tres brokers: kafka1:9092, kafka2:9092 y kafka3:9092, trabajando en modo KRaft.

## Idea clave

---

Kafka recibe, conserva y reparte eventos. El procesamiento analítico complejo se delega a motores como Spark Structured Streaming o Flink.

## 2. Particiones, claves, grupos y offsets

### Topics, particiones y orden

---

Cada partición es un log append-only con offsets 0, 1, 2, ... El orden está garantizado dentro de una partición, pero no existe un orden global entre todas las particiones del topic.

Más particiones permiten mayor paralelismo porque más consumidores pueden trabajar simultáneamente dentro del mismo consumer group.

### La clave decide dónde cae el evento

---

Kafka usa la clave para asignar una partición. La misma clave siempre cae en la misma partición mientras no cambie el número de particiones. Esto conserva el orden relativo de los eventos asociados a esa clave.

El riesgo es una clave caliente: si unas pocas claves concentran gran parte de los eventos, algunas particiones reciben mucha más carga que otras.

### Consumer groups y rebalanceo

---

Dentro de un consumer group, una partición es leída por un solo consumidor a la vez. Si hay más consumidores que particiones, algunos quedan sin trabajo. Si un consumidor se desconecta, Kafka redistribuye sus particiones mediante un rebalanceo.

Consumer groups diferentes son independientes: cada uno puede leer el topic completo con sus propios offsets.

### Offsets, commit y lag

---

El commit indica hasta qué offset el grupo confirma que procesó correctamente. El log-end-offset representa el final actual de la partición. El lag es la diferencia entre ambos.

Si el lag crece de forma sostenida, los consumidores no alcanzan la tasa de producción. La solución puede ser acelerar el procesamiento, aumentar particiones y consumidores o reducir trabajo por evento.

## 3. Replicación, disponibilidad y garantías

### Líder, réplicas e ISR

---

Cada partición tiene un broker líder y réplicas seguidoras. El conjunto ISR contiene las réplicas que se encuentran sincronizadas con el líder.

Con factor de replicación 3, la información de cada partición existe en los tres brokers. Si el líder falla, una réplica en ISR puede asumir el liderazgo.

### acks y min.insync.replicas

---

El productor del práctico utiliza acks=all. Esto exige confirmación suficiente de las réplicas en sincronía. En el clúster, min.insync.replicas es 2, por lo que puede fallar un broker sin perder datos confirmados.

### Garantías de entrega

---

At-most-once: se confirma antes de procesar; puede perder eventos. At-least-once: se procesa y luego se confirma; evita pérdidas, pero puede repetir eventos. Exactly-once: requiere mecanismos adicionales como idempotencia y transacciones.

La estrategia más común es at-least-once con consumidores idempotentes: si el mismo evento se procesa dos veces, el resultado final no cambia.

### Retención e integración con Spark

---

Kafka conserva los mensajes según una política de retención y permite reprocesar el histórico. En el práctico se configuraron 14 días para asegurar que el topic siguiera disponible en el Práctico 5.

Spark Structured Streaming puede leer el topic usando DataFrames y procesar el flujo en micro-batches, agregando manejo de estado, ventanas y tolerancia a fallas.

## 4. Desarrollo del Práctico 4 - Diseño y productor

### Punto de partida

---

Se trabajó en ~/work/practico4 con cuatro piezas principales: g4.py, panel.py, productor.py y consumidor.py. El notebook actuó como panel de control para crear el topic y observar particiones, grupos, offsets y lag.

### Diseño del stream del Grupo 4

---

Evento: una reseña de Amazon Software. Campos enviados: timestamp, parent\_asin, user\_id, rating, helpful\_vote, verified\_purchase y title. Se omitió text porque es el campo más pesado y no era necesario para las métricas.

Tiempo de evento: timestamp, convertido desde milisegundos al formato ISO. Clave: parent\_asin, porque agrupa todas las reseñas del mismo producto y preserva su orden relativo.

Topic: g4.resenas. Configuración: 6 particiones, factor de replicación 3 y retención de 14 días.

### Carga rápida del productor

---

Primera etapa: python productor.py --tasa 0 --max 100000. Resultado real: 100.000 eventos en 5,4 s, equivalentes a 18.361 eventos/s.

Distribución observada: P0=15.406, P1=13.644, P2=20.831, P3=19.178, P4=17.254 y P5=13.687. El reparto no fue perfectamente uniforme por la distribución real de parent\_asin.

### Continuación en vivo

---

Segunda etapa: python productor.py --tasa 200 --desde 100000. Se enviaron 81.675 eventos en 408,4 s, manteniendo 200 eventos/s.

Total final en g4.resenas: 181.675 reseñas desde 2022. El topic se verificó al final con fin(g4.resenas), manteniendo intactos los seis finales de partición.

## 5. Consumidor, métricas y consumer groups

### Consumidor del Grupo 4

---

El consumidor enriqueció cada reseña mediante la tabla auxiliar meta, que relaciona parent\_asin con nombre del producto y tienda.

La métrica en vivo mostró número de reseñas y rating promedio por producto. También se visualizó nombre y tienda.

### Regla de alerta

---

Se implementó la sugerencia del profesor: alerta cuando un producto acumula 10 o más reseñas de 1 estrella durante un mismo día de tiempo de evento.

La regla se validó con casos reales, entre ellos Paramount+ el 2022-02-06 y Stremium el 2022-02-09.

### Dos relojes

---

mirar() permitió distinguir tiempo de evento y tiempo de llegada a Kafka. Para métricas como reseñas por hora debe usarse timestamp, porque representa cuándo ocurrió realmente la reseña; la llegada a Kafka describe la ingestión.

### Reparto entre dos consumidores

---

Con consumidor2 y sebastian dentro de g4.top, Kafka repartió las particiones en dos bloques: [0,1,2] y [3,4,5]. En una medición el lag total fue 380.

Al cerrar consumidor2 con Ctrl-C, Kafka rebalanceó y sebastian recibió [0,1,2,3,4,5]. Luego el grupo alcanzó nuevamente lag 0.

### Segundo grupo independiente

---

Se creó g4.auditoria. Este grupo recibió las seis particiones y alcanzó lag 0 de forma independiente, demostrando que cada consumer group mantiene sus propios offsets.

## 6. Experimento de lag y recuperación

### Objetivo

---

Medir de forma controlada cuánto aumenta el consumer lag cuando el productor sigue enviando eventos y no existe ningún consumidor activo.

### Prueba controlada

---

Se utilizó un topic auxiliar g4.lagtest para no alterar el topic principal. La producción se fijó en 200 eventos/s y se confirmó que el grupo g4.lag estaba Empty antes de medir.

Lag inicial: 39.779. Después de 30,0 s sin consumidores: 45.781. Crecimiento observado: 6.002 eventos.

### Comparación teórica

---

Valor esperado = 200 eventos/s x 30 s = 6.000 eventos. Diferencia real: 2 eventos. El experimento coincidió prácticamente de forma exacta con la teoría.

### Recuperación

---

Al volver a iniciar lag1, el consumidor recibió nuevamente las seis particiones y procesó el atraso hasta alcanzar lag 0.

Una observación importante es que lag() usa offsets confirmados del grupo, mientras el consumidor puede calcular su posición local. Con auto-commit cada 5 s pueden existir diferencias temporales entre ambas mediciones.

### Qué se aprendió al depurar la prueba

---

Dos procesos con el mismo nombre lag1 estaban activos y Kafka dividía las particiones entre ambos. Se cerraron los procesos duplicados antes de repetir.

También se detectó que medir 30 s cuando el productor ya había terminado producía crecimiento 0. La solución fue iniciar la medición solo después de confirmar que realmente estaban entrando nuevos eventos.

## 7. Commit manual, at-least-once e idempotencia

### Primera prueba y corrección

---

El primer intento leyó 1.000 eventos sobre las seis particiones y, al reiniciar, la segunda lectura comenzó por otra partición. La intersección dio 0, por lo que el experimento no demostraba de manera inequívoca la repetición.

Se corrigió usando una única partición y un nuevo group\_id: g4.manual2.

### Experimento definitivo

---

Primera lectura: 1.000 eventos de la partición 0, offsets 0 a 999, con enable\_auto\_commit=False. El consumidor se cerró sin commit para simular una caída.

Al reiniciar el mismo grupo, el commit seguía en None y Kafka volvió a entregar exactamente los offsets 0 a 999. Eventos procesados dos veces: 1.000.

Después del segundo procesamiento se ejecutó commit() y el offset confirmado quedó en 1000.

### Conclusión de la garantía

---

El resultado demuestra at-least-once: Kafka evita perder eventos, pero puede volver a entregarlos si la aplicación cae después de procesarlos y antes de confirmar el offset.

### ¿El consumidor es idempotente?

---

No. El consumidor incrementa contadores, suma ratings y aumenta el conteo de reseñas de 1 estrella. Si un evento se repite, el estado cambia una segunda vez y pueden alterarse métricas o alertas.

Para volverlo idempotente se requeriría deduplicación por identificador único, un estado externo o un motor de streaming con estado y garantías más robustas.

## 8. Resultado final y puntos para explicar

### Entrega final

---

La carpeta work/practico4 quedó limpia con: practico4\_g4\_FINAL.ipynb, productor.py, consumidor.py, g4.py y panel.py.

El topic g4.resenas no se borró, ya que será utilizado por Spark Streaming en el Práctico 5. La verificación final mostró 181.675 eventos distribuidos en las seis particiones.

### Preguntas que el grupo debe poder responder

---

- 1) ¿Por qué parent\_asin es una buena clave? Porque mantiene juntas y ordenadas las reseñas de un mismo producto y permite calcular sus métricas completas en un consumidor.
- 2) ¿Por qué 6 particiones? Porque permite hasta seis consumidores activos en paralelo dentro de un mismo grupo.
- 3) ¿Qué es ISR? El conjunto de réplicas sincronizadas con el líder.
- 4) ¿Qué mide el lag? La diferencia entre el final del log y el último offset confirmado por el grupo.
- 5) ¿Qué ocurre en un rebalanceo? Kafka redistribuye particiones entre consumidores activos.
- 6) ¿Kafka transfiere los contadores en memoria? No; solo particiones y offsets. El estado local de Python no se migra.
- 7) ¿Por qué un segundo grupo puede leer todo otra vez? Porque cada consumer group mantiene offsets independientes.
- 8) ¿Qué significa at-least-once? Que un evento se procesa una o más veces; puede repetirse, pero no debería perderse.
- 9) ¿Por qué nuestro consumidor no es idempotente? Porque procesar dos veces la misma reseña vuelve a modificar contadores y sumas.
- 10) ¿Qué pediríamos a Spark Structured Streaming? Estado distribuido, checkpointing, ventanas, recuperación automática y agregaciones globales.

### Conclusión

---

El Práctico 4 permitió pasar de análisis batch a una arquitectura real de streaming. Más que publicar datos, el objetivo fue entender cómo Kafka distribuye, replica, confirma y recupera eventos, y dónde aparecen las limitaciones de un consumidor Python con estado local.

## 9. Fuentes y referencias del curso

### Material utilizado

---

BIG\_DATA\_KAFKA.pdf - clase Apache Kafka, CDD7018, Dante Travisany.

Practico4-CDD7018-Kafka.pdf - pauta oficial del Práctico 4.

practico4\_g4\_FINAL.ipynb - evidencia y resultados del Grupo 4.

productor.py, consumidor.py, g4.py y panel.py - scripts de ejecución y apoyo del práctico.

### Referencias mencionadas en la clase

---

Kreps, Narkhede y Rao (2011), Kafka: a Distributed Messaging System for Log Processing.

Kreps (2013), The Log: What every software engineer should know about real-time data's unifying abstraction.

Shapira, Palino, Sivaram y Petty (2021), Kafka: The Definitive Guide, 2a edición.

Kleppmann (2017), Designing Data-Intensive Applications, capítulo 11.

### Recordatorio de entrega

---

La pauta indica entregar en work/practico4 un notebook con salidas, junto con productor.py y consumidor.py. Antes de cerrar: detener productores y consumidores con Ctrl-C y no borrar el topic principal.